NEXURA ARCHITECTURE SPECIFICATION

Nexura E-Commerce Platform

Complete Microservices Architecture, API Contract & System Engineering Guide

SYSTEM ARCHITECTURE	FRONTEND FRAMEWORK
Decoupled Microservices	React 18 + Vite SPA
BACKEND MICROSERVICES	PERSISTENCE LAYER
Node.js + Express REST APIs	MySQL 8.0 / SQLite Zero-Config
AUTHENTICATION SYSTEM	STATE MANAGEMENT
JWT + Role-Based Guards	React Context API + Custom Hooks

1. Executive Summary & Core Objectives

Nexura is a production-grade full-stack e-commerce solution architected around domain-driven, decoupled microservices. Designed with an Apple-inspired neo-minimalist aesthetic, the system provides high throughput, fault tolerance, strict domain isolation, and real-time inventory management.

🏢 Strict Domain Isolation

Independent services for user/catalog and orders/cart with separate persistent databases.

⚡ Synchronous Inter-Service REST

Automated stock validation and atomic decrement upon checkout completion.

🔒 Enterprise JWT Security

Role-based authorization (Admin / Customer) with automatic interceptor token refresh.

📦 Hybrid Storage Engine

Dual MySQL production schemas with automatic, zero-config SQLite persistence fallback.

2. System Topology & Architecture

3. Microservice 1: User & Product Service

Port: `5001` | **Database:** `ecommerce_users_products`

This service is the authoritative source for customer profiles, authentication tokens, product taxonomy, and catalog inventory levels.

Database Models & Schemas

- **User:** `id`, `name`, `email`, `password` (bcrypt hashed), `role` (`admin` / `customer`), `phone`, `avatar`, timestamps.
- **Category:** `id`, `name`, `slug`, `description`, `image`.
- **Product:** `id`, `name`, `slug`, `description`, `price`, `discount_price`, `image`, `category`, `category_id`, `stock`, `rating`, `num_reviews`, `is_featured`.

REST Endpoints Reference

Method	Route	Access	Description
POST	`/api/auth/register`	Public	Creates a new customer account and issues JWT.
POST	`/api/auth/login`	Public	Authenticates email/password; returns JWT and user payload.
GET	`/api/auth/me`	Authenticated	Returns profile of current authenticated user.
GET	`/api/products`	Public	Lists products with filters (category, min/max price, sort, search).
GET	`/api/products/:id`	Public	Fetches individual product specifications and stock.
POST	`/api/products/reduce-stock`	Internal / Auth	Inter-service endpoint to deduct stock upon order placement.
POST	`/api/products`	Admin	Creates a new product item in catalog.
PUT	`/api/products/:id`	Admin	Updates existing product properties and stock.
DELETE	`/api/products/:id`	Admin	Deletes product from catalog.
GET	`/api/stats/dashboard`	Admin	Aggregates catalog volume, user counts, and inventory health.

4. Microservice 2: Order & Cart Service

Port: `5002` | **Database:** `ecommerce_orders`

This microservice manages user shopping carts, promo voucher evaluation, checkout orchestration, invoice generation, and status tracking.

Inter-Service Communication Flow

> When an order is created, `Order & Cart Service` executes synchronous HTTP requests to `http://localhost:5001/api/products/:id` to confirm product availability and unit prices. Upon successful checkout, it calls `/api/products/reduce-stock` to automatically decrement inventory in Microservice 1.

REST Endpoints Reference

Method	Route	Access	Description
GET	`/api/cart`	Authenticated	Fetches active user cart with populated item details.
POST	`/api/cart/add`	Authenticated	Adds item or increments quantity in user's cart.
PUT	`/api/cart/items/:id`	Authenticated	Modifies item quantity (1 to Max Stock).
DELETE	`/api/cart/items/:id`	Authenticated	Removes line item from cart.
POST	`/api/orders`	Authenticated	Places order, decrements stock, records transaction invoice.
GET	`/api/orders/my-orders`	Authenticated	Retrieves order history for current logged-in user.
GET	`/api/orders/:id`	Authenticated	Detailed order view including items, delivery, and timeline.
PUT	`/api/orders/:id/status`	Admin	Updates order status (Confirmed ➔ Processing ➔ Shipped ➔ Delivered).

5. Frontend Architecture & Design System

The client interface is built with **React 18** and **Vite**, following a modular component architecture without heavy CSS framework bloat.

Application Page Hierarchy

Page	Route	Audience	Key Features
Home	`/`	All	Hero showcase, category ribbons, featured items carousel.
Catalog	`/products`	All	Search, price filter, category pills, rating sort, grid view.
Product Detail	`/products/:id`	All	Stock counters, reviews list, quantity selector, specs sheet.
Shopping Cart	`/cart`	All	Item removal, quantity controls, promo code `NEXURA10`.
Checkout	`/checkout`	Customer	Address validation, payment method selector, order summary.
Order Tracker	`/orders/:id`	Customer / Admin	4-stage animated progress bar, downloadable invoice breakdown.
Admin Dashboard	`/admin`	Admin	Revenue KPIs, product management modal, order fulfillment controls.

6. Operations & Demo Credentials

Role	Email Address	Password	Permissions
Administrator	`admin@ecommerce.com`	`Admin@123`	Full access to Admin Dashboard, Product CRUD, Order Fulfillment
Customer	`customer@ecommerce.com`	`Customer@123`	Browsing, Cart, Checkout, Order Tracking, Profile Management

Standard Startup Commands

```
# 1. Initialize databases and seed initial demo data
npm run init:db

# 2. Start all microservices and frontend concurrently
npm start

# Access URLs:
# - Frontend Application:    http://localhost:5173
# - User Service API:        http://localhost:5001/api
# - Order Service API:       http://localhost:5002/api
```